Python intermédio – consulta rápida

1 · Iteradores e geradores

01 ITERADORES

cap. 25 · §1

`it = iter([10, 20, 30])`	Criar um iterador a partir de um iterável (lista, texto, dicionário, ficheiro…).
`next(it)  next(it)`	Elemento seguinte: `10` · `20`.
`next(it, None)`	Com valor por omissão: não lança `StopIteration` no fim.
`list(it)  list(it)`	Um iterador só se percorre uma vez: a segunda lista sai vazia.

```
class Contagem:
    def __iter__(self):
        return self
    def __next__(self):
        if self.atual > self.fim:
            raise StopIteration
        self.atual += 1
        return self.atual - 1
```

Iterador numa classe: `__iter__` devolve o iterador; `__next__` devolve o elemento seguinte ou lança `StopIteration`.

O que o `for` faz por dentro

```
it = iter(dados)
while True:
    try:
        x = next(it)
    except StopIteration:
        break
```

Igual a `for x in dados:`. Funciona com qualquer objeto que tenha `__iter__`.

02 FUNÇÕES QUE DEVOLVEM ITERADORES

cap. 11, 19, 25

`enumerate(nomes, start=1)`	Pares (posição, valor).
`zip(nomes, notas)`	Pares (nome, nota), em paralelo.
`map(f, dados)  filter(f, dados)`	Aplicar · filtrar, um elemento de cada vez.
`range(10**9)`	Não cria mil milhões de números: gera-os à medida.
`reversed(lista)  open(f)`	Percorrer ao contrário · um ficheiro linha a linha.

03 GERADORES E YIELD

cap. 25 · §2

```
def contagem(inicio, fim):
    while inicio <= fim:
        yield inicio
        inicio += 1
```

`yield` entrega um valor e pausa a função; retoma no `next` seguinte.

`g = contagem(1, 3)`	Não executa nada ainda: devolve um gerador.
`next(g)  list(g)`	`1` · `[2, 3]` (o que faltava).
`(x ** 2 for x in dados)`	Expressão geradora: como uma list comprehension, com parênteses.
`sum(x ** 2 for x in dados)`	Dentro de uma função, dispensa os parênteses extra.

```
def impares():
    n = 1
    while True:
        yield n
        n += 2
```

Gerador infinito: só produz o que for pedido.

04 ITERTOOLS

cap. 25 · §2

`from itertools import islice, count`	Ferramentas para iteradores (biblioteca padrão).
`list(islice(impares(), 5))`	Os 5 primeiros de qualquer iterador: `[1, 3, 5, 7, 9]`.
`count(10, 10)`	10, 20, 30, … sem fim.

05 LISTA OU GERADOR?

cap. 25 · §2

Lista `[ ]`	Guarda tudo; percorre-se várias vezes; aceita `len`, posições e fatias.
Gerador `( )`	Um valor de cada vez; quase não ocupa memória; percorre-se uma só vez.
Regra prática	Gerador para dados grandes ou infinitos que se percorrem uma vez; lista no resto.

06 ZIP, ANY E ALL

cap. 25 · §3

`for nome, nota in zip(nomes, notas):`	Percorrer listas em paralelo.
`dict(zip(nomes, notas))`	Juntar duas listas num dicionário.
`zip(a, b, strict=True)`	`ValueError` se os tamanhos forem diferentes (sem `strict`, pára no mais curto).
`any(n < 10 for n in notas)`	Pelo menos um verdadeiro?
`all(n >= 10 for n in notas)`	Todos verdadeiros?

07 DECORADORES

cap. 25 · §4

```
import functools

def cronometrar(funcao):
    @functools.wraps(funcao)
    def nova(*args, **kwargs):
        inicio = time.perf_counter()
        res = funcao(*args, **kwargs)
        fim = time.perf_counter()
        print(f"{fim - inicio:.3f} s")
        return res
    return nova
```

Recebe uma função e devolve uma versão com comportamento extra. `wraps` mantém o nome e a docstring. Não esquecer o `return res`.

`@cronometrar` `def treinar(dados): ...`	O mesmo que `treinar = cronometrar(treinar)`.

`def repetir(vezes):`
`  def decorador(funcao): ...`
`  return decorador`
	Decorador com argumentos (`@repetir(3)`): mais um nível de função.

Decoradores que já conheces

`@functools.lru_cache`	Memoização: guarda resultados (cap. 18).
`@property  @x.setter`	Métodos como atributos (cap. 20).
`@dataclass  @abstractmethod`	Classes de dados · métodos obrigatórios (cap. 20).
`@torch.no_grad()`	PyTorch: sem gradientes numa função.

08 GESTORES DE CONTEXTO E WITH

cap. 25 · §5

```
class Cronometro:
    def __enter__(self):
        self.t0 = time.perf_counter()
        return self
    def __exit__(self, tipo, valor, tb):
        fim = time.perf_counter()
        print(f"{fim - self.t0:.3f} s")
        return False
```

`__enter__`: início do bloco; o que devolve vai para o `as`. `__exit__`: corre sempre no fim, mesmo com erro. `False` deixa o erro continuar.

`with Cronometro() as c:` `  treinar(dados)`	«Preparar → usar → arrumar», garantido.

```
from contextlib import contextmanager
@contextmanager
def secao(titulo):
    print("início:", titulo)
    try:
        yield
    finally:
        print("fim")
```

A forma curta: um gerador com um único `yield`. Antes do `yield`: preparar; no `finally`: arrumar.

Gestores de contexto nas bibliotecas

`with torch.no_grad():`	PyTorch: avaliar sem calcular gradientes.

09 ANOTAÇÕES DE TIPO

cap. 25 · §6

`def media(notas: list[float]) -> float:`	Tipos dos parâmetros e do resultado.
`dict[str, int]  tuple[int, int]`	Dicionário · tuplo com dois inteiros.
`int | None  int | float`	Um ou outro (3.10+; antes: `Optional[int]`).
`Iterator[int]  Callable[[int], str]`	Gerador · função (`from collections.abc import …`).
`mypy ficheiro.py`	Verifica os tipos. O Python não os verifica ao correr.